KEYCLOAK DEPLOYMENT RUNBOOK

Complete Step-by-Step | Every Command | Every File | Every Fix

Author	Moiz Aleem — DevOps Intern
Cluster	Red Hat OpenShift on AWS
Namespace	**moizaleem21-dev**
Keycloak Version	26.3.3 (codecentric/keycloakx)
Database	PostgreSQL 18.3 (bitnami/postgresql)
Date	April 11-12, 2026
Final Status	**ALL SYSTEMS RUNNING**

PHASE 0 — Project Folder Setup

First thing — create a clean folder structure. This keeps all files organized.

```
# Create project folders
mkdir -p
keycloak-deployment/{environments/{dev,staging,prod},base,scripts,docs}
cd keycloak-deployment

# Create placeholder files
touch environments/dev/keycloak-official-values.yaml
touch environments/dev/postgres-values.yaml
touch base/keycloak-route.yaml
touch scripts/deploy.sh
```

Final folder structure:

```
keycloak-deployment/
├── base/
│   └── keycloak-route.yaml                # OpenShift Route manifest
├── environments/
│   └── dev/
│       ├── keycloak-official-values.yaml   # Keycloak Helm values
│       └── postgres-values.yaml            # PostgreSQL Helm values
└── scripts/
    └── deploy.sh                           # Deployment script
```

PHASE 1 — Connect to Cluster

1 Login to OpenShift Cluster

Get login token from OpenShift Web Console → top right → Copy Login Command

```
# Login with token from OpenShift web console
oc login --token=sha256~YOUR_TOKEN_HERE --server=https://your-cluster:6443

# Verify you are connected
oc whoami
# Expected output: moizaleem21

# See all nodes in cluster
oc get nodes
```

2 Add Helm Repositories

Add bitnami (for PostgreSQL) and codecentric (for Keycloak) charts

```
# Add Bitnami repo (PostgreSQL charts)
helm repo add bitnami https://charts.bitnami.com/bitnami

# Add Codecentric repo (official Keycloak chart)
helm repo add codecentric https://codecentric.github.io/helm-charts

# Update all repos to get latest versions
helm repo update

# Verify repos added
helm repo list
# Should show: bitnami and codecentric
```

3 Create Namespace

Create dedicated namespace for Keycloak deployment

```
# Create namespace (OpenShift calls it 'project')
oc new-project moizaleem21-dev

# Switch to that namespace
oc project moizaleem21-dev

# Verify current namespace
oc project
# Expected: Using project moizaleem21-dev
```

PHASE 2 — Create Secrets

IMPORTANT: Never put passwords in YAML files. Always use Kubernetes Secrets.

4 Create PostgreSQL Secret

Contains database password — referenced by both PostgreSQL and Keycloak

```
# Create database credentials secret
oc create secret generic keycloak-db-secret \
  --from-literal=postgres-password='StrongAdminPass123!' \
  --from-literal=password='StrongUserPass456!' \
  --from-literal=replication-password='StrongReplPass789!' \
  -n moizaleem21-dev

# Verify secret created (values are hidden)
oc get secret keycloak-db-secret -n moizaleem21-dev
# NAME                  TYPE     DATA   AGE
# keycloak-db-secret    Opaque   3      5s
```

5 Create Keycloak Admin Secret

Admin console login password

```
# Create Keycloak admin credentials secret
oc create secret generic keycloak-admin-secret \
  --from-literal=admin-password='Admin@12345' \
  -n moizaleem21-dev

# To retrieve password later:
oc get secret keycloak-admin-secret \
  -n moizaleem21-dev \
  -o jsonpath='{.data.admin-password}' | base64 -d
echo ''
```

PHASE 3 — Deploy PostgreSQL

Keycloak needs an external database. Never use built-in H2 in production — data is lost on restart.

6 Create PostgreSQL Values File

OpenShift-compatible config — security context disabled

FILE: environments/dev/postgres-values.yaml

```yaml
# ============================================================
# PostgreSQL Values — OpenShift Compatible
# ============================================================

auth:
  database: keycloak
  username: keycloak
  existingSecret: keycloak-db-secret
  secretKeys:
    adminPasswordKey: postgres-password
    userPasswordKey: password
    replicationPasswordKey: replication-password

primary:
  resources:
    requests:
      memory: '256Mi'
      cpu: '250m'
    limits:
      memory: '512Mi'
      cpu: '500m'
  persistence:
    enabled: true
    size: 10Gi
  # FIX: Disable hardcoded UIDs — let OpenShift assign allowed UID
  podSecurityContext:
    enabled: false
  containerSecurityContext:
    enabled: false

readReplicas:
  replicaCount: 0
  podSecurityContext:
    enabled: false
  containerSecurityContext:
    enabled: false

metrics:
  enabled: false
```

7 Deploy PostgreSQL

Run helm install and wait for pod to be ready

```
# Deploy PostgreSQL
helm install keycloak-postgres bitnami/postgresql \
  --namespace moizaleem21-dev \
  --values environments/dev/postgres-values.yaml \
  --wait \
  --timeout 5m

# Watch pod come up
oc get pods -n moizaleem21-dev -w
# Wait for: keycloak-postgres-postgresql-0   1/1   Running

# Check logs to confirm DB ready
oc logs keycloak-postgres-postgresql-0 -n moizaleem21-dev | tail -5
# Good sign: 'database system is ready to accept connections'
```

8 Verify Database

Connect inside pod and confirm keycloak database exists

```
# Connect to database inside pod
oc exec -it keycloak-postgres-postgresql-0 \
  -n moizaleem21-dev -- \
  env PGPASSWORD=$(oc get secret keycloak-db-secret \
    -n moizaleem21-dev \
    -o jsonpath='{.data.password}' | base64 -d) \
  psql -U keycloak -d keycloak -c '\l'

# Expected: keycloak database listed in output
```

PHASE 4 — Deploy Keycloak

Using codecentric/keycloakx chart with official quay.io/keycloak/keycloak image.

NOTE: docker.io is blocked on this cluster. quay.io works. bitnami chart blocked quay.io image. So we use codecentric chart.

9 Create Keycloak Values File

Main configuration file for Keycloak deployment

FILE: environments/dev/keycloak-official-values.yaml

```yaml
# ============================================================
# Keycloak Values — OpenShift Compatible
# Chart: codecentric/keycloakx
# Image: quay.io/keycloak/keycloak:26.3.3
# ============================================================

image:
  repository: quay.io/keycloak/keycloak
  tag: '26.3.3'
  pullPolicy: IfNotPresent

replicas: 1

# Startup command — removed --hostname-strict-https (deleted in v26)
command:
  - '/opt/keycloak/bin/kc.sh'
  - 'start'
  - '--http-enabled=true'
  - '--http-port=8080'
  - '--hostname-strict=false'

# Environment variables
extraEnv: |
  - name: KC_DB
    value: postgres
  - name: KC_DB_URL
    value: jdbc:postgresql://keycloak-postgres-postgresql:5432/keycloak
  - name: KC_DB_USERNAME
    value: keycloak
  - name: KC_DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: keycloak-db-secret
        key: password
  - name: KC_BOOTSTRAP_ADMIN_USERNAME
    value: admin
  - name: KC_BOOTSTRAP_ADMIN_PASSWORD
    valueFrom:
      secretKeyRef:
        name: keycloak-admin-secret
        key: admin-password
  - name: KC_HEALTH_ENABLED
    value: 'true'
  - name: KC_LOG_LEVEL
    value: 'INFO'
  - name: JAVA_OPTS_APPEND
```

```
    value: '-Xms256m -Xmx768m'

# SCC Fix: remove hardcoded UIDs — OpenShift assigns its own
podSecurityContext:
  fsGroup: null
  runAsUser: null
  runAsNonRoot: false

securityContext:
  runAsUser: null
  runAsNonRoot: false
  allowPrivilegeEscalation: false

resources:
  requests:
    cpu: '250m'
    memory: '512Mi'
  limits:
    cpu: '1000m'
    memory: '1Gi'

service:
  type: ClusterIP
  httpPort: 80

# Health probes — Keycloak 26 uses port 9000 for management
# Path: /auth/health  (kc.http-management-relative-path = /auth)
livenessProbe: |
  httpGet:
    path: /auth/health/live
    port: 9000
  initialDelaySeconds: 120
  periodSeconds: 20
  timeoutSeconds: 10
  failureThreshold: 10

readinessProbe: |
  httpGet:
    path: /auth/health/ready
    port: 9000
  initialDelaySeconds: 60
  periodSeconds: 20
  timeoutSeconds: 10
  failureThreshold: 10

startupProbe: |
  httpGet:
    path: /auth/health
    port: 9000
  initialDelaySeconds: 30
  periodSeconds: 20
  timeoutSeconds: 10
  failureThreshold: 60

ingress:
  enabled: false

postgresql:
  enabled: false
```

10 Dry Run First — Always!

Test before real deploy. Catch errors without breaking anything.

```
# Dry run — renders templates, shows what will be deployed
helm install keycloak codecentric/keycloakx \
  --namespace moizaleem21-dev \
  --values environments/dev/keycloak-official-values.yaml \
  --dry-run 2>&1 | grep -E 'Error|error|Warning'

# If no errors — safe to deploy for real
```

11 Deploy Keycloak

Real deployment — watch logs in second terminal

```
# Deploy Keycloak
helm install keycloak codecentric/keycloakx \
  --namespace moizaleem21-dev \
  --values environments/dev/keycloak-official-values.yaml \
  --timeout 10m

# Terminal 1 — watch pod status
oc get pods -n moizaleem21-dev -w

# Terminal 2 — follow logs
oc logs keycloak-keycloakx-0 -n moizaleem21-dev -f

# Good signs in logs:
# 'Keycloak 26.3.3 on JVM started in XX.XXXs'
# 'Listening on: http://0.0.0.0:8080'
# 'Management interface listening on http://0.0.0.0:9000'
```

PHASE 5 — Create OpenShift Route

Route exposes Keycloak to internet with HTTPS. Created as separate YAML — do NOT put in Helm values.

12 Check Service Name

Must use exact service name in Route spec

```
# Find exact service name
oc get svc -n moizaleem21-dev
# Look for keycloak service — name is: keycloak-keycloakx-http

# Also check port name
oc get svc keycloak-keycloakx-http -n moizaleem21-dev -o yaml | grep -A5
ports
# Port name should be: http
```

13 Create Route File

TLS edge termination — HTTPS at router, HTTP inside cluster

FILE: base/keycloak-route.yaml

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: keycloak
  namespace: moizaleem21-dev
  labels:
    app.kubernetes.io/name: keycloak
  annotations:
    haproxy.router.openshift.io/timeout: '300s'
    haproxy.router.openshift.io/h2c: 'true'
spec:
  # host: keycloak.apps.your-cluster.com  # optional custom domain
  to:
    kind: Service
    name: keycloak-keycloakx-http    # exact service name
    weight: 100
  port:
    targetPort: http
  tls:
    termination: edge                # TLS ends at OpenShift router
    insecureEdgeTerminationPolicy: Redirect   # HTTP -> HTTPS auto
  wildcardPolicy: None
```

14 Apply Route

Deploy the Route and get the HTTPS URL

```
# Apply route
```

```
oc apply -f base/keycloak-route.yaml

# Verify route created
oc get route keycloak -n moizaleem21-dev
# NAME      HOST/PORT                                  TLS
# keycloak  keycloak-moizaleem21-dev.apps.cluster.com  edge

# Get full URL
KEYCLOAK_URL=$(oc get route keycloak \
  -n moizaleem21-dev \
  -o jsonpath='{.spec.host}')
echo 'Admin Console: https://'$KEYCLOAK_URL'/admin'
```

PHASE 6 — Verify Everything

15 Full Health Check

Run these commands to confirm everything is working

```
# Check all pods running
oc get pods -n moizaleem21-dev
# Expected:
# keycloak-keycloakx-0              1/1   Running
# keycloak-postgres-postgresql-0    1/1   Running

# Check Helm releases
helm list -n moizaleem21-dev
# NAME                  STATUS    CHART
# keycloak              deployed  keycloakx-2.x.x
# keycloak-postgres     deployed  postgresql-18.x.x

# Check route
oc get route -n moizaleem21-dev

# Check storage
oc get pvc -n moizaleem21-dev
# STATUS should be: Bound

# Check resource usage
oc top pods -n moizaleem21-dev
```

16 Access Admin Console

Open browser and login to Keycloak

```
# Get URL
KEYCLOAK_URL=$(oc get route keycloak -n moizaleem21-dev -o
jsonpath='{.spec.host}')
echo 'Go to: https://'$KEYCLOAK_URL'/admin'
echo 'Username: admin'
echo 'Password: Admin@12345'
```

In browser go to: https://<your-route-url>/admin

Login with: admin / Admin@12345

PHASE 7 — Admin Console Setup (UI)

All done via browser. No CLI needed for this part.

17 Create Realm

Realm = isolated space for your app users. Never use master realm for apps.

- Click top-left dropdown (shows 'Keycloak' or 'master')
- Click 'Create Realm'
- Realm name: aiops-dev
- Display name: AIOps Development
- Enabled: ON
- Click Create

18 Create Roles

Roles control what each user can access

- Left sidebar → Realm roles → Create role
- Create role: aiops-admin (Full platform access)
- Create role: aiops-analyst (Read only dashboard access)
- Create role: ml-engineer (ML model endpoints access)

19 Create Test User

Create a user and assign role

- Left sidebar → Users → Create new user
- Username: testuser | Email: testuser@aiops.com
- Enabled: ON | Email verified: ON | Click Create
- Credentials tab → Set password → Test@12345 → Temporary: OFF → Save
- Role mapping tab → Assign role → select aiops-analyst → Assign

20 Create Client

Client = your application that will use Keycloak for login

- Left sidebar → Clients → Create client
- Client type: OpenID Connect | Client ID: aiops-dashboard
- Client authentication: ON | Standard flow: ON | Direct access grants: ON
- Valid redirect URIs: https://dashboard.apps.cluster.com/*
- Click Save
- Credentials tab → Copy client secret → save it!

21 Security Hardening

Enable brute force protection

- Realm settings → Security defenses tab
- Brute Force Detection: Enabled ON
- Max login failures: 5
- Wait increment: 60 seconds
- Max wait: 900 seconds
- Click Save

TROUBLESHOOTING — All Errors + Fixes

Every error that came during this deployment and exactly how it was fixed.

Error 1 — SCC Forbidden (Pod cannot start)

ERROR	FIX
pods is forbidden: unable to validate against any security context constraint. runAsUser: 1001 must be in range [1007210000, 1007219999]	Set podSecurityContext.enabled: false and containerSecurityContext.enabled: false in values.yaml. OpenShift assigns its own valid UID automatically.

Debug commands used:

```
oc describe pod keycloak-postgres-postgresql-0 -n moizaleem21-dev | grep
-A20 Events
# Shows: 'unable to validate against any security context constraint'

# Tried this first (no cluster-admin access so failed):
oc adm policy add-scc-to-user anyuid -z default -n moizaleem21-dev
# Error: User moizaleem21 cannot get resource securitycontextconstraints

# Fix: in postgres-values.yaml add:
# podSecurityContext:
#   enabled: false
# containerSecurityContext:
#   enabled: false
```

Error 2 — ImagePullBackOff (docker.io blocked)

ERROR	FIX
Failed to pull image docker.io/bitnami/postgresql:16.x: manifest unknown. Mirrors also failed: mirror.gcr.io also failed.	Cluster blocks docker.io completely. Updated helm repo and used latest chart version without pinning. Cluster pulled from approved mirror automatically.

```
# Debug: see exact error
oc get events -n moizaleem21-dev --sort-by=.lastTimestamp | tail -10

# Fix: update repo and reinstall without pinned version
helm repo update bitnami
helm uninstall keycloak-postgres -n moizaleem21-dev
helm install keycloak-postgres bitnami/postgresql \
  --namespace moizaleem21-dev \
  --values environments/dev/postgres-values.yaml
  # No --version flag = uses latest available image
```

Error 3 — Bitnami Blocks Non-Standard Image

ERROR	FIX
INSTALLATION FAILED: Original containers have been substituted for unrecognized ones. Unrecognized images: quay.io/keycloak/keycloak:26.3.3	Bitnami v25+ chart only allows Bitnami-signed images. Switched to codecentric/keycloakx chart which has no such restriction.

```
# This error came when trying to use quay.io image with bitnami chart
helm install keycloak bitnami/keycloak \
  --set image.registry=quay.io \
  --set image.repository=keycloak/keycloak
# ERROR: Unrecognized images: quay.io/keycloak/keycloak:26.3.3

# Fix: switch to codecentric chart entirely
helm repo add codecentric https://codecentric.github.io/helm-charts
helm repo update
helm search repo codecentric/keycloakx --versions | head -5
```

Error 4 — CrashLoopBackOff (Unknown flag)

ERROR	FIX
Unknown option: --hostname-strict-https. Try kc.sh start --help for available options.	This flag was removed in Keycloak 26.x. Removed it from command section in values.yaml. Also deleted PVC to clear cached bad config saved to disk.

```
# Debug: get logs from crashed pod
oc logs keycloak-keycloakx-0 -n moizaleem21-dev --previous
# Shows: Unknown option: --hostname-strict-https

# Fix step 1: remove flag from values.yaml command section
# Remove this line:  - '--hostname-strict-https=false'

# Fix step 2: delete PVC (keycloak saved bad config to disk)
helm uninstall keycloak -n moizaleem21-dev
oc delete pod keycloak-keycloakx-0 -n moizaleem21-dev --force
--grace-period=0
oc delete pvc -l app.kubernetes.io/instance=keycloak -n moizaleem21-dev

# Fix step 3: redeploy fresh
helm install keycloak codecentric/keycloakx \
  --namespace moizaleem21-dev \
  --values environments/dev/keycloak-official-values.yaml
```

Error 5 — Pod 0/1 Ready (Health Probe Failing)

ERROR	FIX
Startup probe failed: HTTP probe failed with statuscode: 404. Also: dial tcp: connect: connection refused on port 8080.	Keycloak 26 moved health endpoints to port 9000 (management interface). Path prefix is /auth/health. Updated all probes to use port 9000 and path /auth/health.

```
# Debug step 1: check what probes are configured
oc get pod keycloak-keycloakx-0 -n moizaleem21-dev \
  -o jsonpath='{.spec.containers[0].readinessProbe}' | python3 -m json.tool
# Shows port 8080 — WRONG for Keycloak 26

# Debug step 2: check open ports (no curl/wget in image)
oc exec keycloak-keycloakx-0 -n moizaleem21-dev -- \
  cat /proc/net/tcp6 | awk '{print $2}' | cut -d: -f2 | sort -u | \
  while read hex; do
    printf '%d\n' '0x'$hex 2>/dev/null
  done | sort -un
# Shows: 8080 and 9000 both open

# Debug step 3: check management path setting
oc exec keycloak-keycloakx-0 -n moizaleem21-dev -- \
  /opt/keycloak/bin/kc.sh show-config 2>/dev/null | grep -i health
# Shows: kc.http-management-relative-path = /auth

# Debug step 4: test health path using /dev/tcp
oc exec keycloak-keycloakx-0 -n moizaleem21-dev -- /bin/bash -c '
  exec 3<>/dev/tcp/localhost/9000
  printf "GET /auth/health HTTP/1.0\r\nHost: localhost\r\n\r\n" >&3
  head -1 <&3'
# Shows: HTTP/1.0 200 OK — correct path confirmed

# Fix: update values.yaml probes to port 9000 and /auth/health path
# Then full reinstall (helm upgrade does not update StatefulSet probes)
helm uninstall keycloak -n moizaleem21-dev
oc delete pvc -l app.kubernetes.io/instance=keycloak -n moizaleem21-dev
helm install keycloak codecentric/keycloakx \
  --namespace moizaleem21-dev \
  --values environments/dev/keycloak-official-values.yaml
```

Error 6 — extraDeploy Route Not Created

ERROR	FIX
Route not appearing after adding extraDeploy section in keycloak-official-values.yaml.	codecentric/keycloakx chart does not support extraDeploy properly. Solution: create Route as a separate YAML file and apply with oc apply.

```
# Create separate route file
# File: base/keycloak-route.yaml  (see Phase 5)

# Apply it separately
oc apply -f base/keycloak-route.yaml

# Verify
oc get route -n moizaleem21-dev
```

Error 7 — Route HTTP Only, No HTTPS

ERROR	FIX
Route created but only HTTP access. Browser shows 'Not Secure'.	Route was missing the tls section. Added tls.termination: edge and insecureEdgeTerminationPolicy: Redirect to route spec.

```
# Delete old route
oc delete route keycloak -n moizaleem21-dev

# Add TLS section to base/keycloak-route.yaml:
# spec:
#   tls:
#     termination: edge
#     insecureEdgeTerminationPolicy: Redirect

# Reapply
oc apply -f base/keycloak-route.yaml

# Verify TLS
oc get route keycloak -n moizaleem21-dev
# TERMINATION column should show: edge
```

QUICK REFERENCE — Most Used Commands

Status Checks

```
oc get pods -n moizaleem21-dev                          # all pods + status
oc get pods -n moizaleem21-dev -w                       # watch live
oc get all -n moizaleem21-dev                           # everything
oc get route,svc,pvc -n moizaleem21-dev                # networking +
storage
oc top pods -n moizaleem21-dev                          # CPU/memory usage
helm list -n moizaleem21-dev                            # helm releases
```

Debugging

```
oc describe pod <podname> -n moizaleem21-dev            # full detail +
events
oc logs <podname> -n moizaleem21-dev                    # current logs
oc logs <podname> -n moizaleem21-dev --previous         # logs before crash
oc logs <podname> -n moizaleem21-dev -f                 # follow live
oc logs <podname> -n moizaleem21-dev --tail=50          # last 50 lines
oc get events -n moizaleem21-dev --sort-by=.lastTimestamp  # recent events
```

Pod Shell & Exec

```
oc exec -it <podname> -n moizaleem21-dev -- /bin/bash  # open shell in pod
oc exec <podname> -n moizaleem21-dev -- <command>      # run single command

# Check open ports (no curl/wget needed)
oc exec <pod> -n moizaleem21-dev -- cat /proc/net/tcp6

# HTTP test without curl (bash built-in)
oc exec <pod> -n moizaleem21-dev -- /bin/bash -c '
  exec 3<>/dev/tcp/localhost/9000
  printf "GET /auth/health HTTP/1.0\r\nHost: localhost\r\n\r\n" >&3
  head -1 <&3'
```

Helm Operations

```
helm install <n> <chart> -f values.yaml --dry-run      # test first!
helm install <n> <chart> -f values.yaml                # real deploy
helm upgrade <n> <chart> -f values.yaml                # update
helm uninstall <n> -n moizaleem21-dev                  # remove
helm history <n> -n moizaleem21-dev                    # all versions
helm rollback <n> -n moizaleem21-dev                   # instant rollback
helm list -n moizaleem21-dev                           # show releases
```

Secrets

```
# Read a secret value
oc get secret <name> -n moizaleem21-dev \
  -o jsonpath='{.data.<key>}' | base64 -d && echo

# Get DB password
oc get secret keycloak-db-secret -n moizaleem21-dev \
  -o jsonpath='{.data.password}' | base64 -d && echo

# Get admin password
oc get secret keycloak-admin-secret -n moizaleem21-dev \
  -o jsonpath='{.data.admin-password}' | base64 -d && echo
```

Cleanup (Full Reset)

```
# Remove all Keycloak resources
helm uninstall keycloak -n moizaleem21-dev
helm uninstall keycloak-postgres -n moizaleem21-dev
oc delete route keycloak -n moizaleem21-dev
oc delete pvc --all -n moizaleem21-dev
oc delete pod --all -n moizaleem21-dev --force --grace-period=0
```

FINAL DEPLOYMENT CHECKLIST

✔	Cluster connected — oc login successful
✔	Helm repos added — bitnami and codecentric
✔	Namespace created — moizaleem21-dev
✔	Secrets created — keycloak-db-secret and keycloak-admin-secret
✔	postgres-values.yaml created with SCC fix
✔	PostgreSQL deployed — pod 1/1 Running
✔	Database verified — keycloak DB accessible via psql
✔	keycloak-official-values.yaml created — correct image, probes, SCC fix
✔	Keycloak deployed via codecentric/keycloakx chart
✔	Keycloak pod 1/1 Running — started in ~15 seconds
✔	keycloak-route.yaml created with TLS edge termination
✔	Route applied — oc apply -f base/keycloak-route.yaml
✔	HTTPS URL accessible in browser
✔	Admin console login working — admin/Admin@12345
✔	aiops-dev realm created
✔	Roles created — aiops-admin, aiops-analyst, ml-engineer
✔	Test user created and role assigned
✔	Client created — aiops-dashboard
✔	Brute force protection enabled

DEPLOYMENT COMPLETE

Keycloak 26.3.3 | PostgreSQL 18.3 | OpenShift | moizaleem21-dev

Completed by Moiz Aleem — DevOps Intern — April 2026